JAVA PROGRAMMING

Paper Code: ETCS-307	**L**	**T/P**	**C**
Paper: Java Programming	**3**	**1**	**4**

INSTRUCTIONS TO PAPER SETTERS: **MAXIMUM MARKS: 75**

1. Question No. 1 should be compulsory and cover the entire syllabus. This question should have objective or short answer type questions. It should be of 25 marks.
2. Apart from Question No. 1, rest of the paper shall consist of four units as per the syllabus. Every unit should have two questions. However, student may be asked to attempt only 1 question from each unit. Each question should be of 12.5 marks

Objective: To learn object oriented concepts and enhancing programming skills.

UNIT I

Overview and characteristics of Java, Java program Compilation and Execution Process Organization of the Java Virtual Machine, JVM as an interpreter and emulator, Instruction Set, class File Format, Verification, Class Area, Java Stack, Heap, Garbage Collection. Security Promises of the JVM, Security Architecture and Security Policy. Class loaders and security aspects, sandbox model

[T1,R2][No. of Hrs.: 11]

UNIT II

Java Fundamentals, Data Types & Literals Variables, Wrapper Classes, Arrays, Arithmetic Operators, Logical Operators, Control of Flow, Classes and Instances, Class Member Modifiers Anonymous Inner Class Interfaces and Abstract Classes, inheritance, throw and throws clauses, user defined Exceptions, The String Buffer Class, tokenizer, applets, Life cycle of applet and Security concerns.

[T1,T2][No. of Hrs.: 12]

UNIT III

Threads: Creating Threads, Thread Priority, Blocked States, Extending Thread Class, Runnable Interface, Starting Threads, Thread Synchronization, Synchronize Threads, Sync Code Block, Overriding Synced Methods, Thread Communication, wait, notify and notify all.

AWT Components, Component Class, Container Class, Layout Manager Interface Default Layouts, Insets and Dimensions, Border Layout, Flow Layout, Grid Layout, Card Layout Grid Bag Layout AWT Events, Event Models, Listeners, Class Listener, Adapters, Action Event Methods Focus Event Key Event,Mouse Events, Window Event

[T2][No. of Hrs.: 11]

UNIT IV

Input/Output Stream, Stream Filters, Buffered Streams, Data input and Output Stream, Print Stream Random Access File, JDBC (Database connectivity with MS-Access, Oracle, MS-SQL Server), Object serialization, Sockets, development of client Server applications, design of multithreaded server. Remote Method invocation, Java Native interfaces, Development of a JNI based application.

Collection API Interfaces, Vector, stack, Hashtable classes, enumerations, set, List, Map, Iterators.

[T1][R1][No. of Hrs.: 10]

Text Books:

[T1] Patrick Naughton and Herbertz Schidt, “Java-2 the complete Reference”,TMH
[T2] Sierra & bates, “Head First Java”, O’reilly

Reference Books:

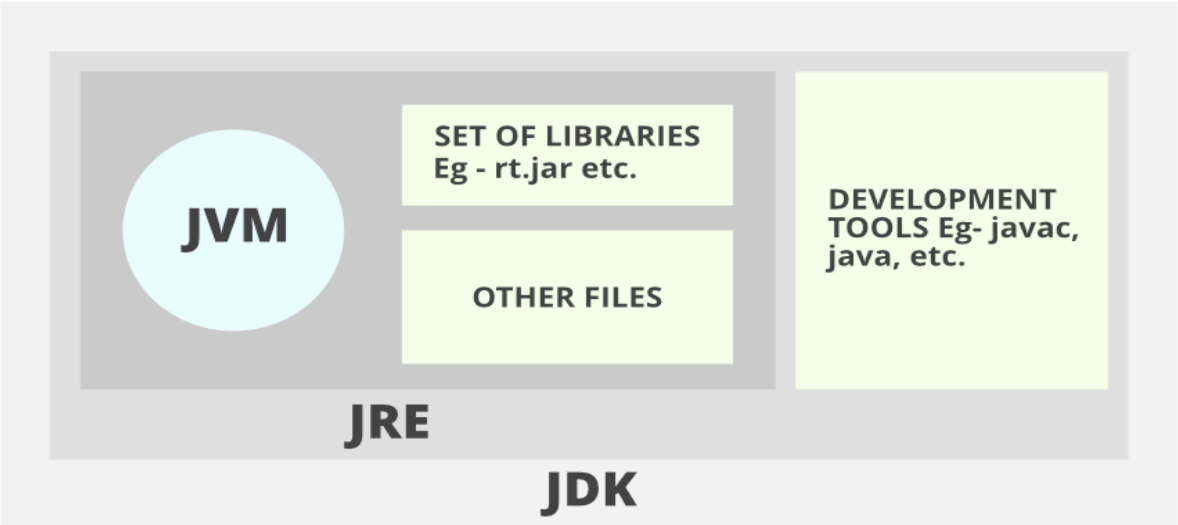
[R1] E. Balaguruswamy, “Programming with Java”, TMH
[R2] Horstmann, “Computing Concepts with Java 2 Essentials”, John Wiley.
[R3] Decker & Hirshfield, “Programming.Java”, Vikas Publication.

Difference Between JDK and JRE in Java

JDK and JRE are the core concepts in Java programming and their differences are some of the most popular interview questions. We don't use these concepts while programming but if we want to become a Java developer, we must know about these concepts.

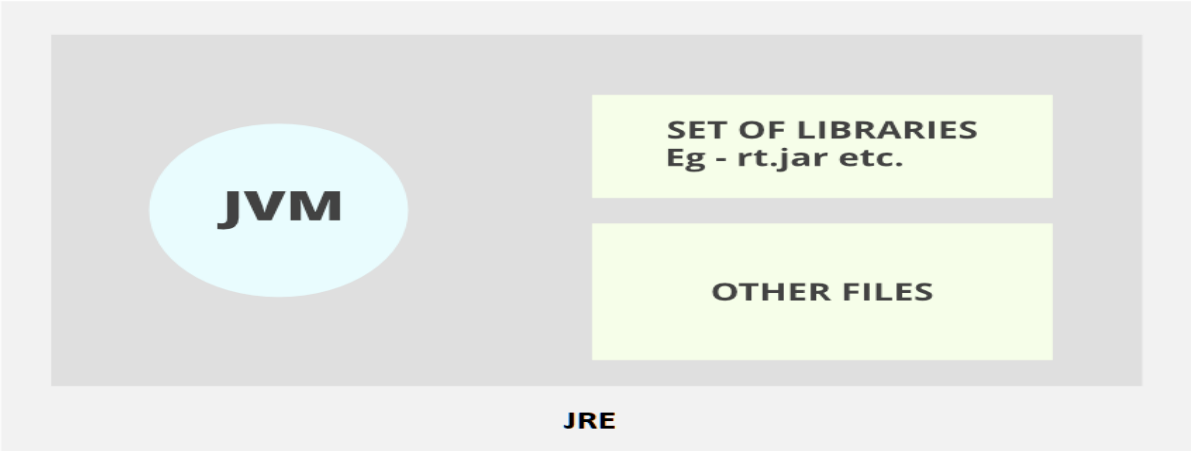
JDK

JDK stands for Java Development Kit. It is a software development environment used to develop Java applications and applets. It is a platform-specific software i.e. there are separate installers for Windows, Mac, and Unix systems. Java developers can use it on their Windows, macOS, Solaris, and Linux to code and run Java programs. It contains Java Runtime Environment(JRE) and other development tools like an interpreter, compiler, archiver, and a document generator. We can possibly install more than one JDK version on the same computer.

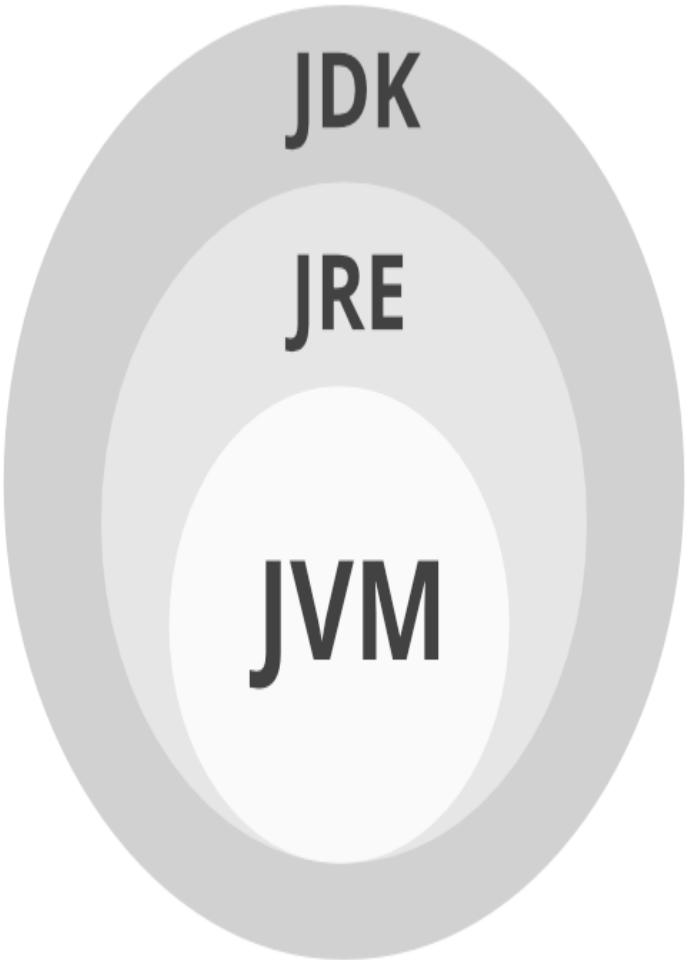


JRE

JRE stands for Java Runtime Environment. It is the implementation of JVM (Java Virtual Machine) and it is specially designed to provide an environment to execute Java programs. It is also platform dependent like JDK. It consists of JVM, Java binaries, and other classes for the smooth execution of the program. It doesn't contain any development tools like a compiler, debugger, etc. If we only want to execute a program, we just need to install JRE and not JDK, since there is no development or compilation of the code required.



Let’s see the main difference with the help of the below diagram:



Difference Between JDK and JRE

KEY	JDK	JRE
DEFINITION	JDK(Java Development Kit) is used to develop Java applications. JDK also contains numerous development tools like compilers, debuggers, etc.	JRE(Java Runtime Environment) is the implementation of JVM(Java Virtual Machine) and it is specially designed to execute Java programs.
FUNCTIONALITY	It is mainly used for the execution of code and its main functionality is development.	It is mainly used for creating an environment for code execution.
DEPENDENCY OF PLATFORM	It is platform-dependent.	It is also platform-dependent like JDK.
TYPE OF TOOLS	Since JDK is responsible for the development purpose, therefore it contains tools which are required for development and	On the other hand, JRE is not responsible for development purposes so it doesn't contain such tools as the compiler, debugger, etc. Instead, it contains class libraries and

	debugging purpose.	supporting files required for the purpose of execution of the program.
IMPLEMENTATION OF JDK AND JRE	JDK = JRE + other development tools.	JRE = JVM + other class libraries.

Java Runtime Environment

The Java Runtime Environment (JRE) is software that Java programs require to run correctly. Java is a computer language that powers many current web and

mobile applications. The JRE is the underlying technology that communicates between the Java program and the operating system. It acts as a translator and facilitator, providing all the resources so that once you write Java software, it runs on any operating system without further modifications.

Why is the JRE important?

A software program needs a runtime environment that provides access to memory and other system resources such as program files and dependencies. In the past, most software used the operating system directly as its runtime environment. However, this meant that developers had to write different code for each operating system that they wanted their applications to run on. The Java Runtime Environment (JRE) technology was created as a solution to this problem.

The JRE is actually one of three Java platform components that are required for any Java program to run successfully. The Java Development Kit (JDK) and Java Virtual Machine (JVM) are the other two components.

Java Development Kit

The JDK is a collection of software tools that you can use for developing Java applications. You can set up the JDK in your development environment by downloading and installing it. Select the JDK software version that matches the Java version you want to use. For example, Java Standard Edition, or Java SE, requires the Java SE JDK.

Java Virtual Machine

The JVM is software that runs the Java program line by line. Developers configure the JVM settings to manage program resources when the Java application runs. For example, you can change the JVM memory setting and check how much internal memory your Java applications use at runtime.

Role of the JRE in Java programming language

The JRE combines the Java code that you create by using the JDK with additional built-in code called libraries. It then creates a JVM instance, or local copy, that finally runs the Java programs. JVMs are available for multiple operating systems, and the JRE generates a single copy of your Java code that runs on all types of JVMs. In this way, the JRE facilitates platform

independence for Java applications. You can write them once and run them anywhere.

Difference between the JRE, JVM, and JDK

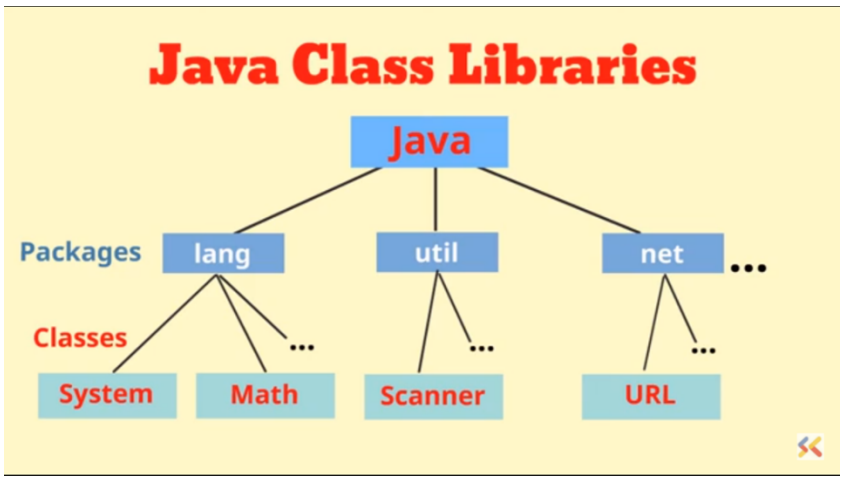
The JDK is a software layer above the JRE that contains a compiler, a debugger, and other tools commonly found in any software development environment. You write code in English-like syntax in the JDK. The JDK compiles it and passes the byte code to the JRE. In contrast, the JRE contains class libraries, supporting files, and the JVM. It uses these software components to run the byte code on any device.

How does the JRE work?

The Java Runtime Environment (JRE) runs on top of the operating system, providing additional Java-specific resources. The Java Development Kit (JDK) and JRE interact to create a sustainable runtime environment that runs Java program files on any machine. The JRE uses three core components to work.

ClassLoader

Java class libraries contain collections of pre-written code that you can call as needed. They simplify the job of Java developers by providing built-in methods for common and non-trivial tasks (Nontrivial is a favourite word among programmers and computer people for describing **any task that is not quick and easy to accomplish**. It may mean "extremely" difficult and time consuming. Therefore, since many programmers are unbridled optimists, take the word seriously!) such as taking input from users, displaying output to users, and more. All Java programs reference several class libraries. The Java ClassLoader dynamically loads all class files necessary into the Java Virtual Machine (JVM) on demand.



Bytecode verifier

The JDK has a compiler that converts the English-like code you write into a machine language version called Java bytecode. They bytecode verifier in the JRE checks the format and accuracy of the Java code before loading it into the JVM. For example, if the code violates system integrity or access rights, the JRE will not load the class file.

Interpreter

After the bytecode successfully loads, the Java interpreter creates the JVM instance that runs the Java program on the underlying machine.

Java programming paradigms

Java programming concept possesses all the advanced programming features that the recent programming languages hold, in addition to these it holds something special which makes it the first of its kind. Following sections are to brief the major characteristics of Java programming.

Object Oriented Programming

Java is an object Oriented Programming (OOP) language. It incorporates almost every OOP features. Object creation through object template i.e. class, data abstraction and encapsulation, data and code sharing through inheritance, overloading concept through polymorphism, and data/process hiding etc. are some basic OOP features in Java, Java is a bit like C++ but more simpler than aristocrat C++. In fact, it is elegant language wherein the poorly understood, erratic, and redundant features of C++ have been eliminated. There are no business of pointers in Java - only lightly bounded heterogeneous collection of data/object. Absence of any kind of pointers means free from huge bugs because pointers manipulation is one of the richest source of bugs in almost all programs. Java is sophisticated enough to help programmers to express complex ideas . Here, global data and stand-alone function are not possible, again, unlike C/C++. The basic programming components in Java is objects which directly or indirectly decedent from the forefathers of classes. The class definition in Java permits both static and dynamic binding and therefore full reuse of code and data sharing. Java is truly Object Oriented Programming in the sense that many class definition can be inherited dynamically. However, Java does not support multiple inheritance whose semantic and usage has been quite controversial. Type casting or operator overloading is also not allowed in Java.

Platform Independent Programming Environment

The microprocessor that is inside every computer can understand and execute only the machine codes i.e. in terms of 0s and 1s. But using high level programming, programmers use source code (English like) and using a translator, code is converted into the machine code relevant to the microprocessor. As the machine codes varies from microprocessor to microprocessor so what we need is that for same source code but for different microprocessor type different translator. Hence, a program (source code) when converted into the machine code for an Intel chip will vary from the version for a Motorola chip, which again will be different from the version for Sun workstation. The same problem exists in case of computers having same microprocessors but different operating systems.

Java has overcome this problem of architecture and environment dependency. Java solves this problem using the concept what is called Java Virtual Machine (JVM). This JVM specifies wonderfully detailed with a dummy CPU and dummy instruction set that looks like standard assembly code. The Java compiler transforms the source programs into the instructions of JVM. The compiled Java code in known as byte code instruction (which are assembly code according to JVM). Next, Java interpreter is there which is specific to each environment (processor and OS) and converts the byte code instructions into the native processor instructions before executing it. Thus, converting a program source into an object code is compile half way and interpret half way. **Figure 1.1** illustrates this mechanism.

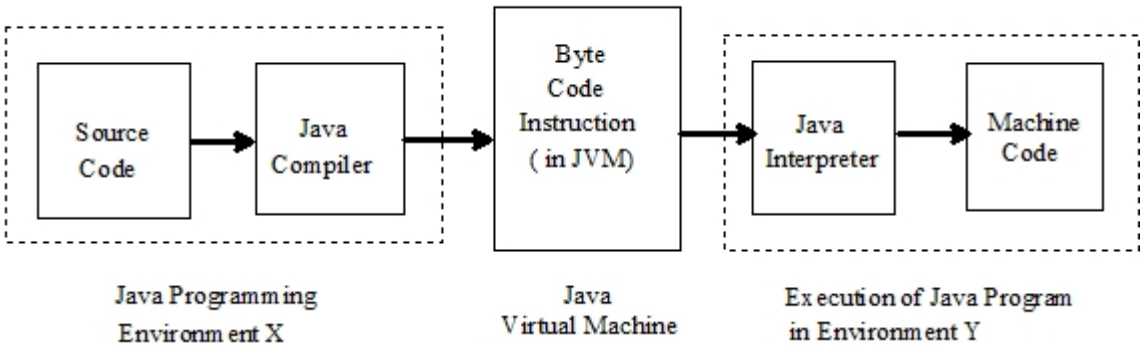


Figure 1.1 : Compilation of Java source code into object code

As the Java byte codes are architecture and operating system independent they are highly portable and can be executed on any system without change. This platform independence is illustrated in **Figure 1.2**

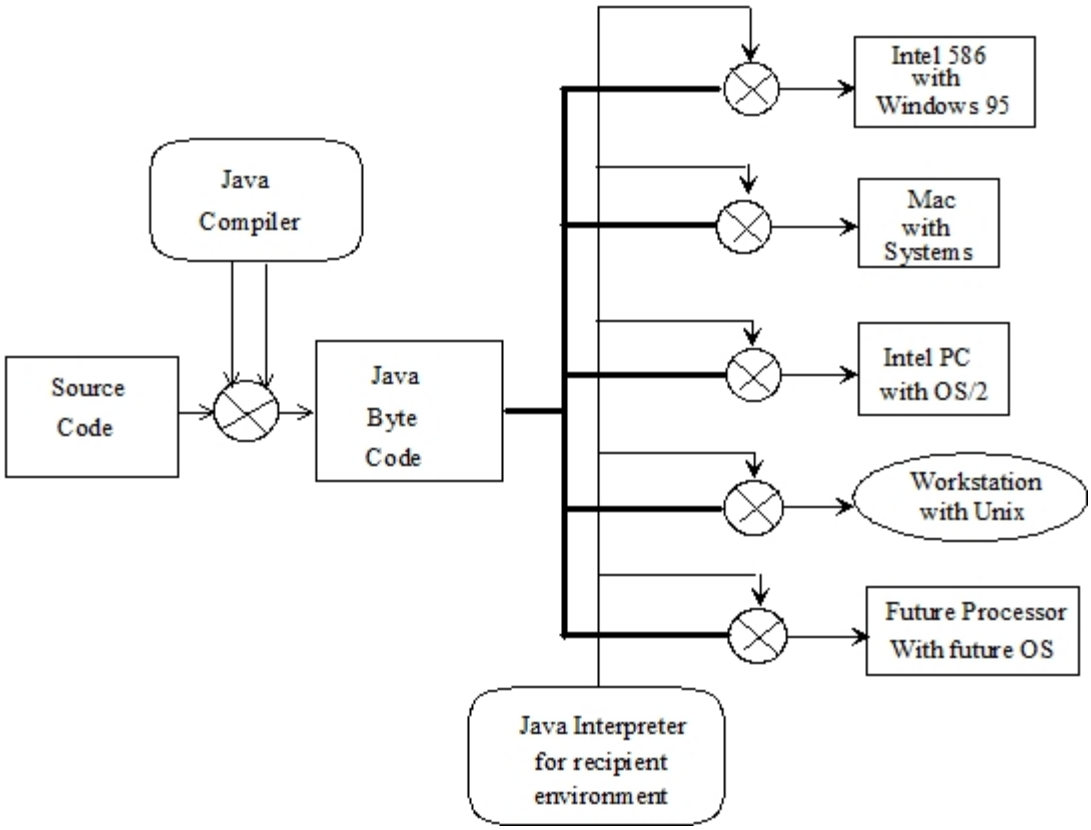


Figure 1.2: Platform independent Java programming

Using the conventional programming languages like C or Pascal, if we need a slice of memory we have to allocate it and then to returns it to the system we are finished with it, Thus, here memory management is solely decided by the programmer. This overhead is eliminated in Java. Here, a program does not need to be malicious to suck up memory and not release it. Java controls all of the memory, Java$\diamond$s run time system supports automatic garbage collectors based memory management system. Every once in a while, it will scan through all of the objects in a process to see if any of them are not used. The obsolete objects are reclaimed from the memory. As there no explicit memory manipulation is required by the programmer, so Java programs are highly robust

Multithreading

Another important contribution of Java is the support of multithreading. The multithreading is basically, the ability to run several tasks within the context of one large application. In more simpler version, a program is a list of instructions , and path through list of instructions is often called the thread of execution. In many cases, execution of a program through multiple threads makes more sense. For an example, suppose, a program needs to open a file, draw some thing on the screen, and print out something else. Each of these may require waiting for access to some resource. A multithreaded program can, in effect, start up three different threads of executions for these tasks. If the printing

thread is waiting for more paper to be loaded in the printer, then the file thread and drawing thread can continue operating. In Java multithreading interpreter can keep track of all threads and switch among all of them, giving each a fair share of the processor's attention. For most applications, multithreading improves interactive performance and real time system development. The multithreading encourages Java to interface and support several features of modern operating systems and network protocols.

Security in Java

Java is based on client /sever technology, After the accomplishment of source code into byte code translation, Java programs are in effect down loaded from host machines and then run on one's client machine. Since Java programs are down loaded from all over the world, some precautions are needed to prevent malafide pieces of code called computer viruses embedded within the programs, causing problems to the recipient system and spreading it around. This therefore demands that distributed applications have to exhibit the highest level of security concerns. A byte code verifier in the Java interpreter looks at the incoming byte codes and verifies whether the compiled code is strictly language compliant, in case it founds illegal code, the run time system rejects the code and refuses to run it; thus trapping all malafide codes. Once the strict verification is over, the application proceeds to run and by this Java impotents security holes if any.

Java Interpreter

Java is a platform-independent programming language. It means that we can run Java on the platforms that have a **Java interpreter**. It is the reason that makes the Java platform-independent. The Java interpreter converts the Java bytecode (.class file) into the code understand by the operating system.

In this section, we will understand **what is an interpreter in Java, the features of the interpreter,** and **how does the Java interpreter work.** We will also see **how it is different from a compiler.**

Q. How many types of memory areas are allocated by JVM?

JVM (Java Virtual Machine) is an abstract machine, In other words, it is a program/software which takes Java bytecode and converts the byte code (line by line) into machine understandable code.

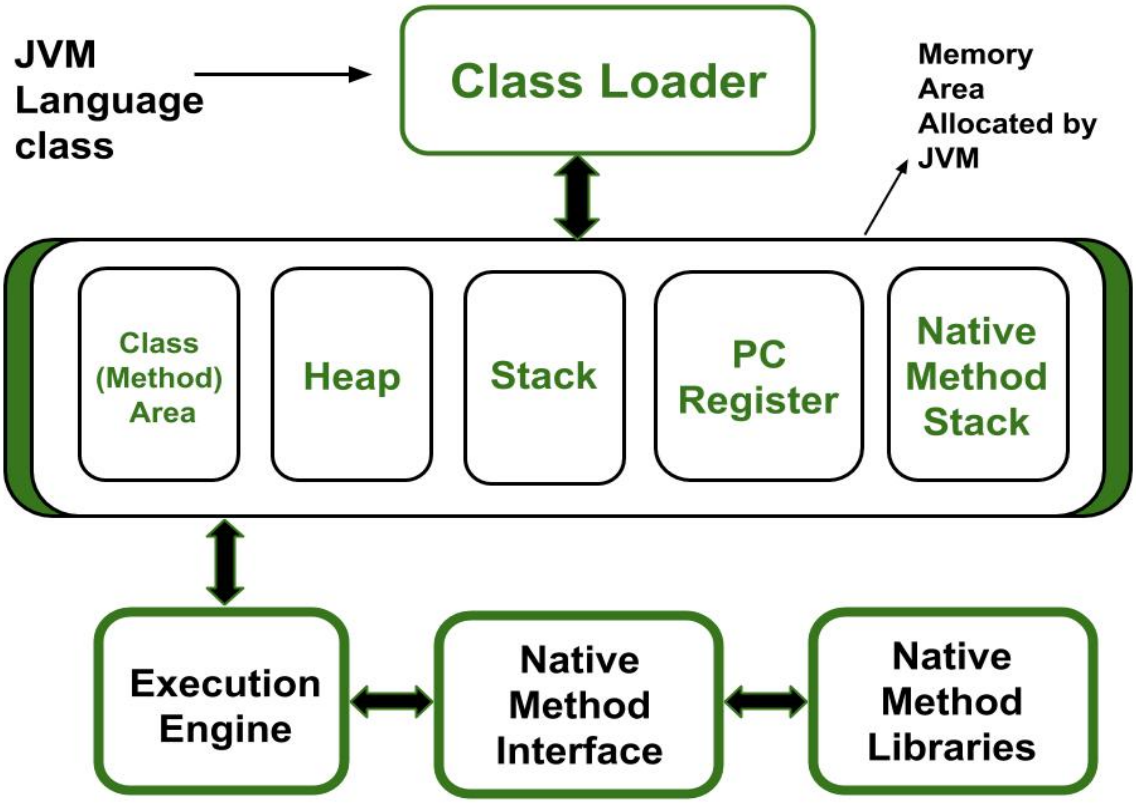
JVM(Java Virtual Machine) acts as a run-time engine to run Java applications. JVM is the one that actually calls the main method present in Java code. JVM is a part of the JRE(Java Runtime Environment).
JVM perform some particular **types of operations**:
1. Loading of code
2. Verification of code
3. Executing the code
4. It provides a run-time environment to the users

ClassLoader
It is a subsystem of JVM which is used to load class files. It is mainly responsible for three activities.

- Loading
- Linking
- Initialization

Types of Memory Areas Allocated By the JVM:

All these functions take different forms of memory structure. The **memory in the JVM is divided into 5 different parts**:



1. Class(Method) Area
2. Heap
3. Stack
4. Program Counter Register

5. Native Method Stack
Let's see about them in brief:

1. Class (Method) Area

The class method area is the memory block that stores the class code, variable code(static variable, runtime constant), method code, and the constructor of a Java program. (Here method means the function which is written inside the class). It stores class-level data of every class such as the runtime constant pool, field and method data, the code for methods.

2. Heap

The Heap area is the memory block where objects are created or objects are stored. Heap memory allocates memory for class interfaces and arrays (an array is an object). It is used to allocate memory to objects at run time

3. Stack

Each thread has a private JVM stack, created at the same time as the thread. It is used to store data and partial results which will be needed while returning value for method and performing dynamic linking.

Java Stack stores frames and a new frame is created each time at every invocation of the method. A frame is destroyed when its method invocation completes

4. Program Counter Register:

Each JVM thread that carries out the task of a specific method has a program counter register associated with it. The non-native method has a PC that stores the address of the available JVM instruction whereas, in a native method, the value of the program counter is undefined. PC register is capable of storing the return address or a native pointer on some specific platform.

5. Native method Stacks:

Also called C stacks, native method stacks are not written in Java language. This memory is allocated for each thread when it's created And it can be of a fixed or dynamic nature.

Q. How many types of memory areas are allocated by JVM?

Different ans:

Java **V**irtual **M**achine is a program/software which takes Java bytecode (.class files)

and converts the byte code (line by line) into machine understandable code.

JVM contains a module known as a class loader. A class loader in JVM loads, links and, initializes a program. It−

- Loads the class into the memory.
- Verifies the byte code instructions.

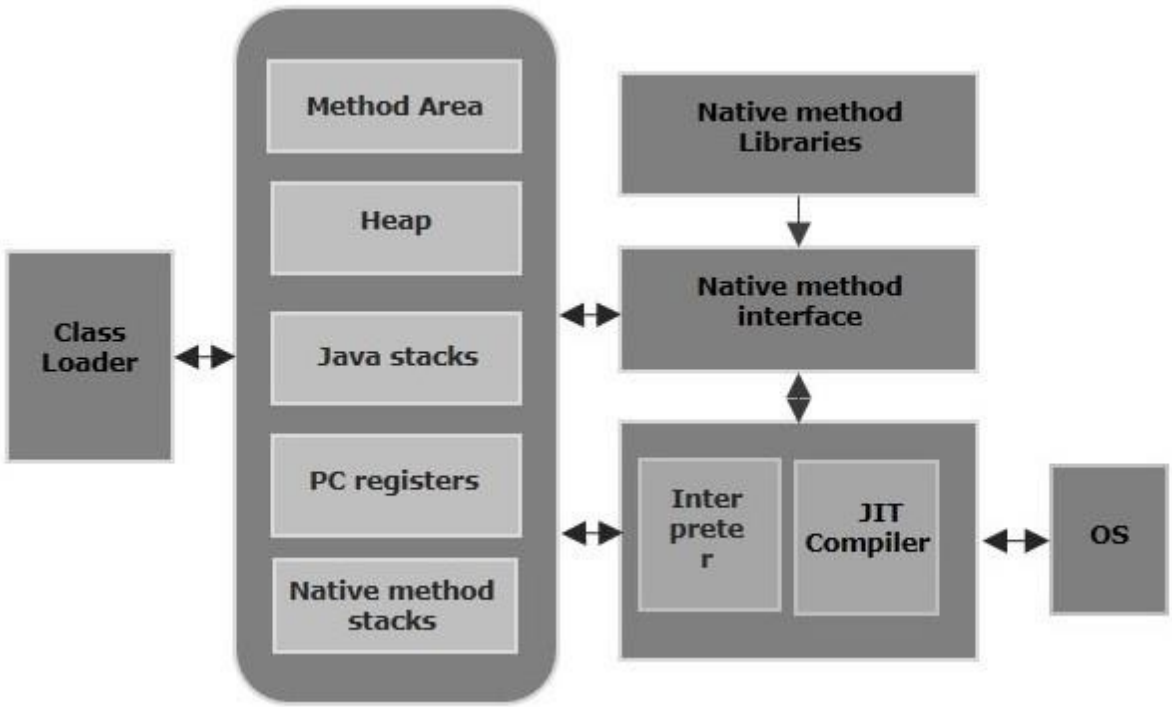
- Allocates memory for the program.

The memory in the JVM is divided into five different parts namely− **Method area−** The method area stores the class code − code of the variables and methods.

Heap − The Java objects are created in this area.

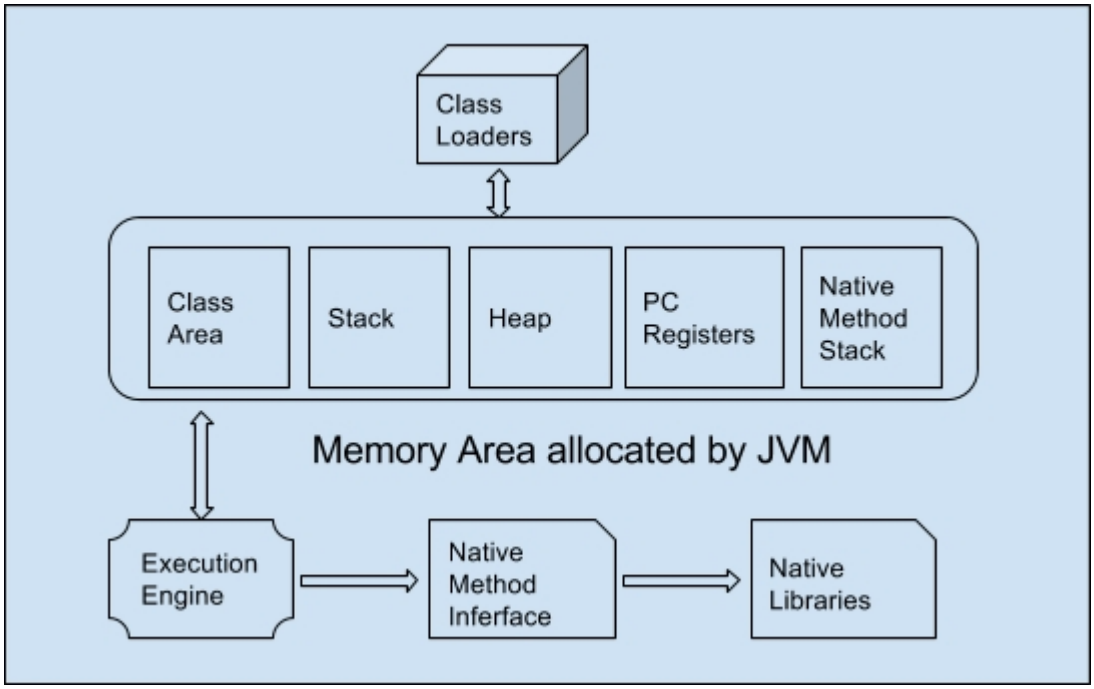
Java Stack− While running methods the results are stored in the stack memory. PC registers− These contain the address of the instructions of the methods.

Native method stacks− Similar to Java stack, native methods are executed on the Native method stacks.



Different way answer

- **Classloader** − Loads the class file into the JVM.
- **Class Area** − Storage areas for a class elements structure like fields, method data, code of method etc.
- **Heap** − Runtime storage allocation for objects.
- **Stack** − Storage for local variables and partial results. A stack contains frames and allocates one for each thread. Once a thread gets completed, this frame also gets destroyed. It also plays roles in method invocation and returns.
- **PC Registers** − Program Counter Registers contains the address of an instruction that JVM is currently executing.
- **Execution Engine** − It has a virtual processor, interpreter to interpret bytecode instructions one by one and a JIT, just in time compiler.
- **Native method stack** − It contains all the native methods used by the a



Garbage Collection in Java

Garbage collection in Java is the process by which Java programs perform automatic memory management. Java programs compile to bytecode that can be run on a Java Virtual Machine, or JVM for short. When Java programs run on the JVM, objects are created on the heap, which is a portion of memory dedicated to the program. Eventually, some objects will no longer be needed. The garbage collector finds these unused objects and deletes them to free up memory.

Q. How Does Garbage Collection in Java works?

Java garbage collection is an automatic process. Automatic garbage collection is the process of looking at heap memory, identifying which objects are in use and which are not, and deleting the unused objects. An in-use object, or a referenced object, means that some part of your program still maintains a pointer to that object. An unused or unreferenced object is no longer referenced by any part of your program. So the memory used by an unreferenced object can be reclaimed. The programmer does not need to mark objects to be deleted explicitly. The garbage collection implementation lives in the JVM.

Types of Activities in Java Garbage Collection

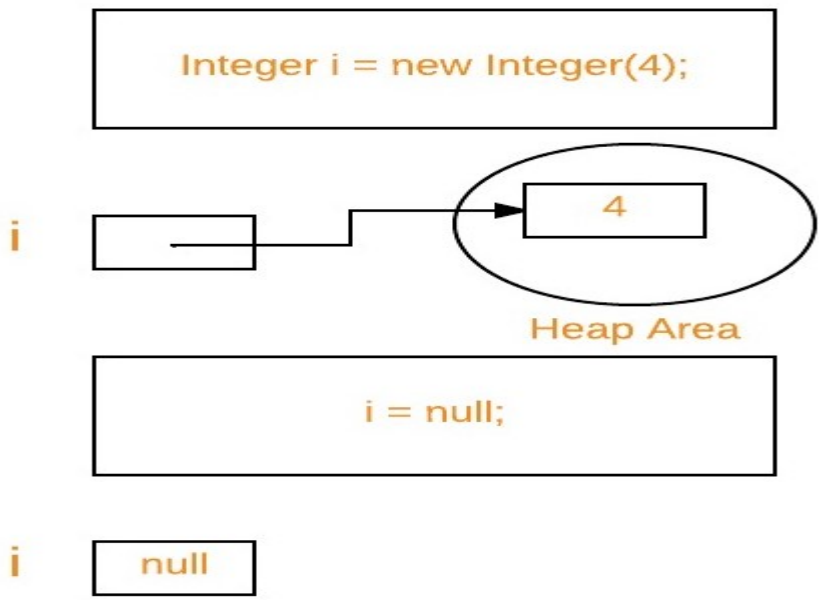
Two types of garbage collection activity usually happen in Java. These are:

1. **Minor or incremental Garbage Collection:** It is said to have occurred when unreachable objects in the young generation heap memory are removed.
2. **Major or Full Garbage Collection:** It is said to have occurred when the objects that survived the minor garbage collection are copied into the old generation or permanent generation heap memory are removed. When compared to the young generation, garbage collection happens less frequently in the old generation.

Important Concepts Related to Garbage Collection in Java

1. Unreachable objects: An object is said to be unreachable if it doesn't contain any reference to it. Also, note that objects which are part of the island of isolation are also unreachable.

```
Integer i = new Integer(4);

// the new Integer object is reachable  via the reference in 'i'

i = null;

// the Integer object is no longer reachable.
```



2. Eligibility for garbage collection: An object is said to be eligible for GC(garbage collection) if it is unreachable. After *i* = *null*, integer object 4 in the heap area is suitable for garbage collection in the above image.

Ways to make an object eligible for Garbage Collector

- Even though the programmer is not responsible for destroying useless objects but it is highly recommended to make an object unreachable(thus eligible for GC) if it is no longer required.
- There are generally four ways to make an object eligible for garbage collection.
  1. Nullifying the reference variable
  2. Re-assigning the reference variable
  3. An object created inside the method
  4. Island of Isolation

*Ways for requesting **JVM** to run Garbage Collector*

- Once we make an object eligible for garbage collection, it may not destroy immediately by the garbage collector. Whenever JVM runs the Garbage Collector program, then only the object will be destroyed. But when JVM runs Garbage Collector, we can not expect.
- We can also request JVM to run Garbage Collector. There are two ways to do it :
  1. **Using *System.gc()* method:** System class contain static method *gc()* for requesting JVM to run Garbage Collector.
  2. **Using *Runtime.getRuntime().gc()* method:** Runtime class allows the application to interface with the JVM in which the application is running. Hence by using its gc() method, we can request JVM to run Garbage Collector.
  3. There is no guarantee that any of the above two methods will run Garbage Collector.
  4. The call *System.gc()* is effectively equivalent to the call : *Runtime.getRuntime().gc()*

Finalization

- Just before destroying an object, Garbage Collector calls *finalize()* method on the object to perform cleanup activities. Once *finalize()* method completes, Garbage Collector destroys that object.
- *finalize()* method is present in Object class with the following prototype.

```
protected void finalize() throws Throwable
```

Based on our requirement, we can override *finalize()* method for performing our cleanup activities like closing connection from the database.

1. The *finalize()* method is called by Garbage Collector, not JVM. However, Garbage Collector is one of the modules of JVM.
2. Object class *finalize()* method has an empty implementation. Thus, it is recommended to override the *finalize()* method to dispose of system resources or perform other cleanups.
3. The *finalize()* method is never invoked more than once for any object.
4. If an uncaught exception is thrown by the *finalize()* method, the exception is ignored, and the finalization of that object terminates.

Advantages of Garbage Collection in Java

The advantages of Garbage Collection in Java are:

- It makes java memory-efficient because the garbage collector removes the unreferenced objects from heap memory.
- It is automatically done by the garbage collector (a part of JVM), so we don’t need extra effort.
  Source:

Method Overriding in Java

1. **Understanding the problem without method overriding**

2. **Can we override the static method**

3. **Method overloading vs. method overriding**

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.

In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an IS-A relationship (inheritance).

```
4. //Java Program to demonstrate why we need method overriding
5. //Here, we are calling the method of parent class with child
6. //class object.
7. //Creating a parent class
8. class Vehicle{
9.   void run(){System.out.println("Vehicle is running");}
10.}
11.//Creating a child class
12.class Bike extends Vehicle{
13.  public static void main(String args[]){
14.  //creating an instance of child class
15.  Bike obj = new Bike();
```

```
16. //calling the method with child class instance
17. obj.run();
18. }
19.}
```

Example of method overriding

In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method are the same, and there is IS-A relationship between the classes, so there is method overriding.

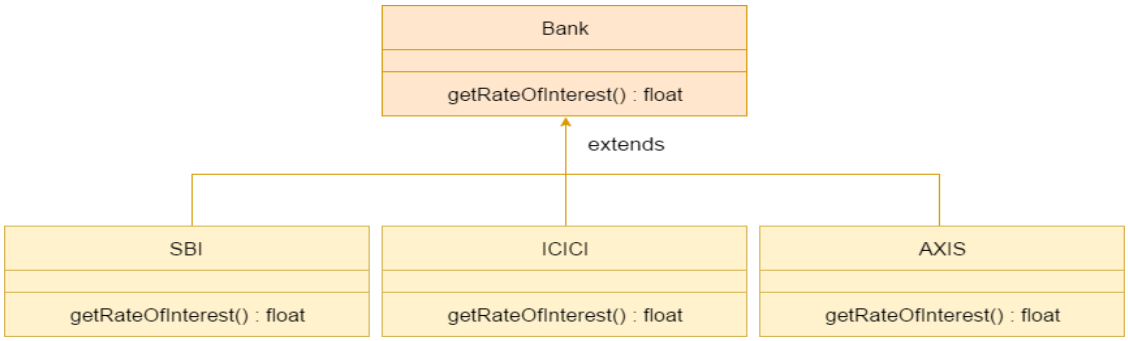
```
1. //Java Program to illustrate the use of Java Method Overriding
2. //Creating a parent class.
3. class Vehicle{
4.   //defining a method
5.   void run(){System.out.println("Vehicle is running");}
6. }
7. //Creating a child class
8. class Bike2 extends Vehicle{
9.   //defining the same method as in the parent class
10.  void run(){System.out.println("Bike is running safely");}
11.
12.  public static void main(String args[]){
13.  Bike2 obj = new Bike2();//creating object
14.  obj.run();//calling method
15.  }
16. }
```

17. Output:

```
18.  Bike is running safely
```

A real example of Java Method Overriding

Consider a scenario where Bank is a class that provides functionality to get the rate of interest. However, the rate of interest varies according to banks. For example, SBI, ICICI and AXIS banks could provide 8%, 7%, and 9% rate of interest.



Java method overriding is mostly used in Runtime Polymorphism which we will learn in next pages.

```
//Java Program to demonstrate the real scenario of Java Method Overriding
//where three classes are overriding the method of a parent class.
//Creating a parent class.
class Bank{
int getRateOfInterest(){return 0;}
}
//Creating child classes.
class SBI extends Bank{
int getRateOfInterest(){return 8;}
}

class ICICI extends Bank{
int getRateOfInterest(){return 7;}
}
class AXIS extends Bank{
int getRateOfInterest(){return 9;}
}
//Test class to create objects and call the methods
```

```
19. class Test2{
20. public static void main(String args[]){
21. SBI s=new SBI();
22. ICICI i=new ICICI();
23. AXIS a=new AXIS();
24. System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
25. System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
26. System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
27. }
28. }
```

Test it Now

```
Output:
SBI Rate of Interest: 8
ICICI Rate of Interest: 7
AXIS Rate of Interest: 9
```

Access Modifiers in Java

1. Private access modifier
2. Role of private constructor
3. Default access modifier
4. Protected access modifier
5. Public access modifier
6. Access Modifier with Method Overriding

There are two types of modifiers in Java: **access modifiers** and **non-access modifiers**.

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

1. **Private**: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

2. **Default**: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected**: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public**: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

There are many non-access modifiers, such as static, abstract, synchronized, native, volatile, transient, etc. Here, we are going to learn the access modifiers only.

Â

Fullscreen

X

Access Modifier	within class	within package	outside package by subclass only	outside pac
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Understanding Java Access Modifiers

Let's understand the access modifiers in Java by a simple table.

1) Private

The private access modifier is accessible only within the class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

```
1. class A{
2. private int data=40;
3. private void msg(){System.out.println("Hello java");}
4. }
5.
6. public class Simple{
7.  public static void main(String args[]){
8.    A obj=new A();
9.    System.out.println(obj.data);//Compile Time Error
10.   obj.msg();//Compile Time Error
11.   }
12. }
```

Role of Private Constructor

If you make any class constructor private, you cannot create the instance of that class from outside the class. For example:

```
1. class A{
2. private A(){}//private constructor
3. void msg(){System.out.println("Hello java");}
4. }
5. public class Simple{
6.  public static void main(String args[]){
7.    A obj=new A();//Compile Time Error
```

```
8.  }
9. }
```

Note: A class cannot be private or protected except nested class.

2) Default

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

```
1. /save by A.java
2. package pack;
3. class A{
4.   void msg(){System.out.println("Hello");}
5. }
```

```
1. //save by B.java
2. package mypack;
3. import pack.*;
4. class B{
5.   public static void main(String args[]){
6.    A obj = new A();//Compile Time Error
7.    obj.msg();//Compile Time Error
8.   }
9. }
```

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

3) Protected

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifer.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
1. //save by A.java
2. package pack;
3. public class A{
4. protected void msg(){System.out.println("Hello");}
5. }
```

```
1. //save by B.java
2. package mypack;
3. import pack.*;
4.
5. class B extends A{
6.   public static void main(String args[]){
7.    B obj = new B();
8.    obj.msg();
9.   }
10.}
```

```
Output:Hello
```

4) Public

The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Example of public access modifier

```
1. /save by A.java
2.
3. package pack;
4. public class A{
5. public void msg(){System.out.println("Hello");}
```

```
6. }
```

```
1. //save by B.java
2.
3. package mypack;
4. import pack.*;
5.
6. class B{
7.   public static void main(String args[]){
8.    A obj = new A();
9.    obj.msg();
10.  }
11.}
```

```
Output:Hello
```

Java Access Modifiers with Method Overriding

If you are overriding any method, overridden method (i.e. declared in subclass) must not be more restrictive.

```
1. class A{
2. protected void msg(){System.out.println("Hello java");}
3. }
4.
5. public class Simple extends A{
6. void msg(){System.out.println("Hello java");}//C.T.Error
7.  public static void main(String args[]){
8.    Simple obj=new Simple();
9.    obj.msg();
10.   }
11.}
```

The default modifier is more restrictive than protected. That is why, there is a compile-time error.

Inheritance in Java

- Difficulty Level : Easy
- Last Updated : 02 Sep, 2022

Inheritance is an important pillar of OOP(Object-Oriented Programming). It is the mechanism in java by which one class is allowed to inherit the features(fields and methods) of another class.

Important terminology:

- **Super Class:** The class whose features are inherited is known as a superclass(or a base class or a parent class).
- **Sub Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

How to use inheritance in Java

The keyword used for inheritance is **extends**.

Syntax :

```
class derived-class extends base-class
{
   //methods and fields
}
```

Example: In the below example of inheritance, class Bicycle is a base class, class MountainBike is a derived class that extends Bicycle class and class Test is a driver class to run program.

- Java

```
// Java program to illustrate the
// concept of inheritance



// base class
```

```
class Bicycle {

    // the Bicycle class has two fields

    public int gear;

    public int speed;



    // the Bicycle class has one constructor

    public Bicycle(int gear, int speed)

    {

        this.gear = gear;

        this.speed = speed;

    }



    // the Bicycle class has three methods

    public void applyBrake(int decrement)

    {

        speed -= decrement;

    }



    public void speedUp(int increment)

    {

        speed += increment;
```

```
    }

    // toString() method to print info of Bicycle
    public String toString()
    {
        return ("No of gears are " + gear + "\n"
                + "speed of bicycle is " + speed);
    }
}

// derived class
class MountainBike extends Bicycle {

    // the MountainBike subclass adds one more field
    public int seatHeight;

    // the MountainBike subclass has one constructor
    public MountainBike(int gear, int speed,
                        int startHeight)
    {
```

```
        // invoking base-class(Bicycle) constructor
        super(gear, speed);
        seatHeight = startHeight;
    }

    // the MountainBike subclass adds one more method
    public void setHeight(int newValue)
    {
        seatHeight = newValue;
    }

    // overriding toString() method
    // of Bicycle to print more info
    @Override public String toString()
    {
        return (super.toString() + "\nseat height is "
                + seatHeight);
    }
}

// driver class
```

```
public class Test {

    public static void main(String args[])

    {



        MountainBike mb = new MountainBike(3, 100, 25);

        System.out.println(mb.toString());

    }

}
```

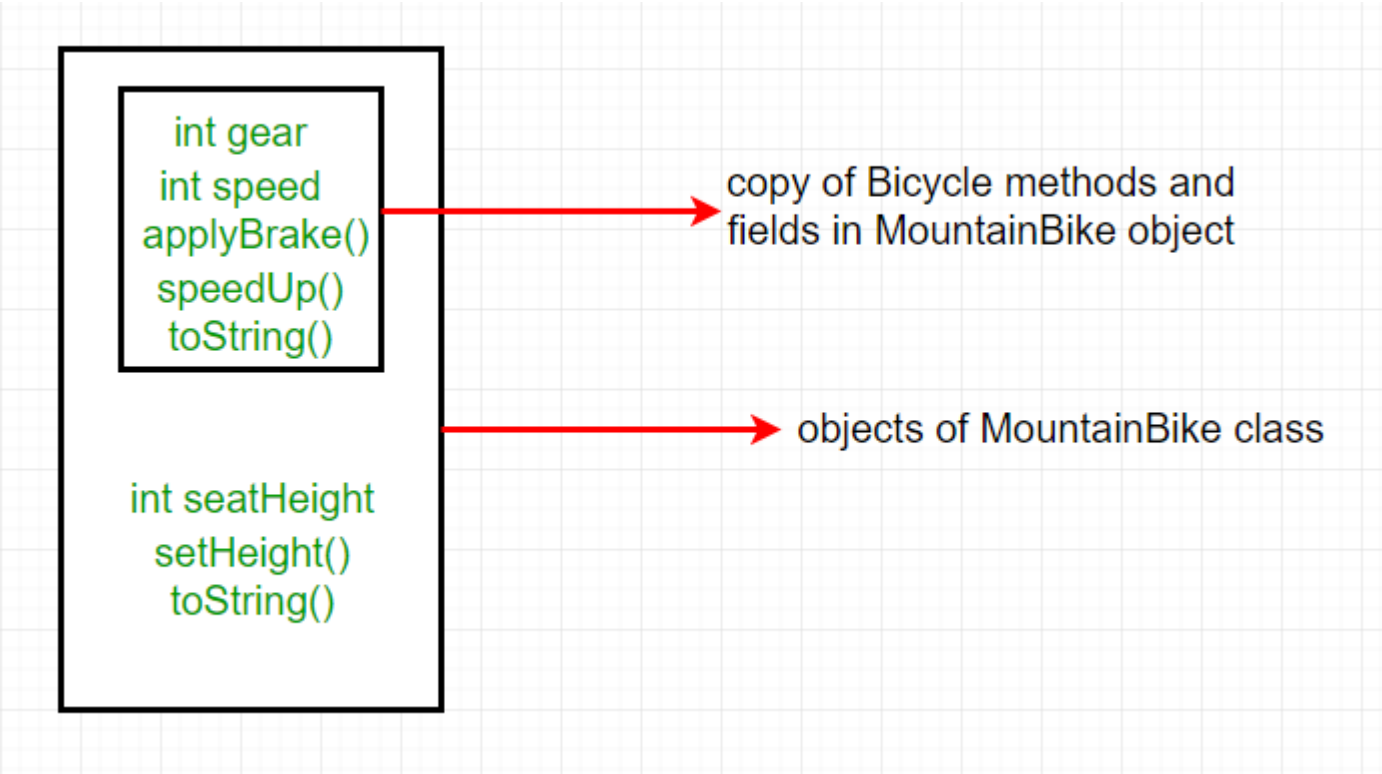
Output

```
No of gears are 3

speed of bicycle is 100

seat height is 25
```

In the above program, when an object of MountainBike class is created, a copy of all methods and fields of the superclass acquire memory in this object. That is why by using the object of the subclass we can also access the members of a superclass.

Please note that during inheritance only the object of the subclass is created, not the superclass. For more, refer Java Object Creation of Inherited Class.

Illustrative image of the program:



In practice, inheritance and polymorphism are used together in java to achieve fast performance and readability of code.

Types of Inheritance in Java

Below are the different types of inheritance which are supported by Java.

1. Single Inheritance: In single inheritance, subclasses inherit the features of one superclass. In the image below, class A serves as a base class for the derived class B.

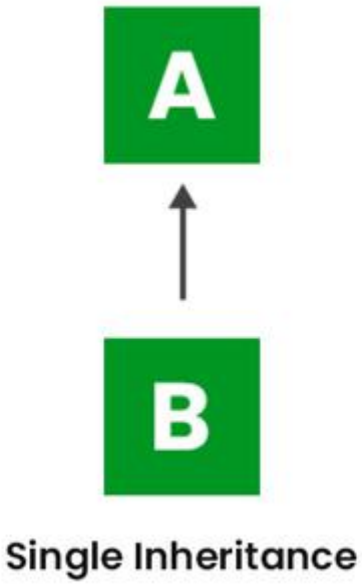
```
/ Java program to illustrate the

// concept of single inheritance

import java.io.*;

import java.lang.*;

import java.util.*;



class one {

    public void print_geek()
```

```
    {
        System.out.println("Geeks");
    }
}


class two extends one {
    public void print_for() { System.out.println("for"); }
}
// Driver class
public class Main {
    public static void main(String[] args)
    {
        two g = new two();
        g.print_geek();
        g.print_for();
        g.print_geek();
    }
}
```

Output

```
Geeks
for
```

```
Geeks
```



Single inheritance

2. Multilevel Inheritance: In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also act as the base class to other class. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the grandparent's members.

- Java

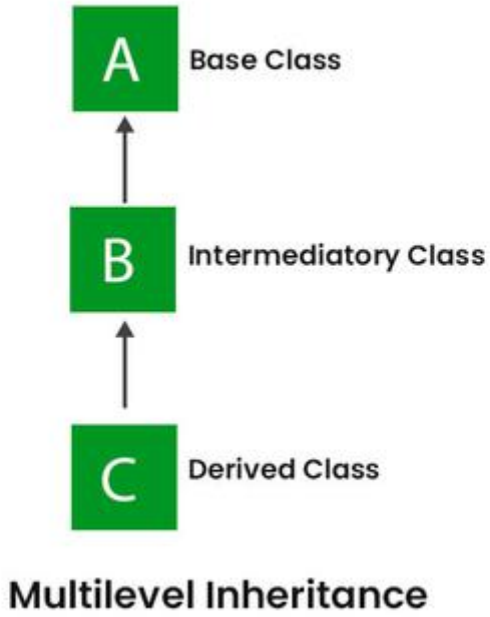
```
// Java program to illustrate the

// concept of Multilevel inheritance

import java.io.*;

import java.lang.*;

import java.util.*;




class one {

    public void print_geek()
```

```
    {
        System.out.println("Geeks");
    }
}


class two extends one {
    public void print_for() { System.out.println("for"); }
}


class three extends two {
    public void print_geek()
    {
        System.out.println("Geeks");
    }
}


// Drived class
public class Main {
    public static void main(String[] args)
    {
```

```
        three g = new three();

        g.print_geek();

        g.print_for();

        g.print_geek();

    }

}
```

Output

```
Geeks
for
Geeks
```



Multilevel Inheritance

3. Hierarchical Inheritance: In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived class B, C and D.

- Java

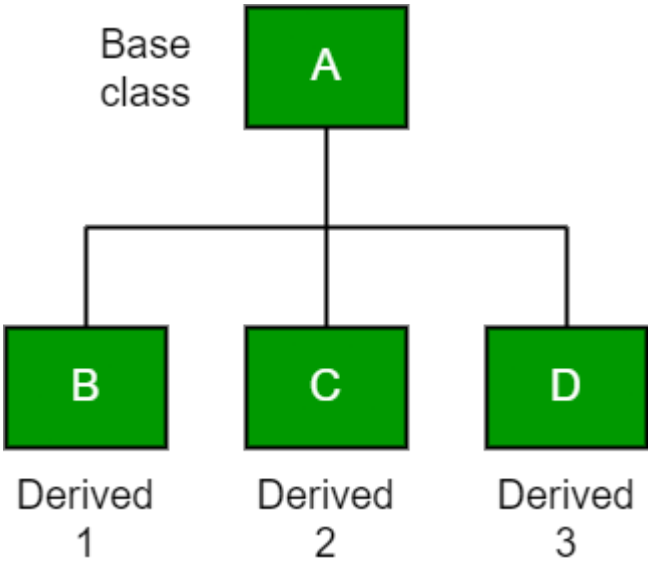
```
// Java program to illustrate the
```

```
// concept of Hierarchical  inheritance


class A {

    public void print_A() { System.out.println("Class A"); }

}


class B extends A {

    public void print_B() { System.out.println("Class B"); }

}


class C extends A {

    public void print_C() { System.out.println("Class C"); }

}


class D extends A {

    public void print_D() { System.out.println("Class D"); }

}


// Driver Class

public class Test {
```

```
    public static void main(String[] args)

    {

        B obj_B = new B();

        obj_B.print_A();

        obj_B.print_B();



        C obj_C = new C();

        obj_C.print_A();

        obj_C.print_C();



        D obj_D = new D();

        obj_D.print_A();

        obj_D.print_D();

    }

}
```

Output

```
Class A
Class B
Class A
Class C
Class A
Class D
```



Hierarchical Inheritance

4. <u>Multiple Inheritance</u> (Through Interfaces): In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes. Please note that Java does **not** support <u>multiple inheritances</u> with classes. In java, we can achieve multiple inheritances only through <u>Interfaces</u>. In the image below, Class C is derived from interface A and B.

- Java

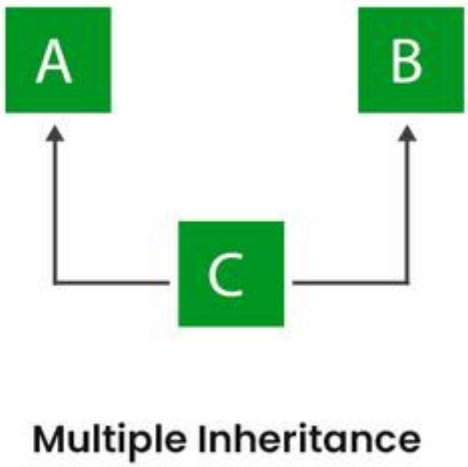
```
// Java program to illustrate the

// concept of Multiple inheritance

import java.io.*;

import java.lang.*;

import java.util.*;



interface one {

    public void print_geek();
```

```
}


interface two {
    public void print_for();
}


interface three extends one, two {
    public void print_geek();
}
class child implements three {
    @Override public void print_geek()
    {
        System.out.println("Geeks");
    }


    public void print_for() { System.out.println("for"); }
}


// Drived class
public class Main {
```

```
    public static void main(String[] args)

    {

        child c = new child();

        c.print_geek();

        c.print_for();

        c.print_geek();

    }

}
```

Output

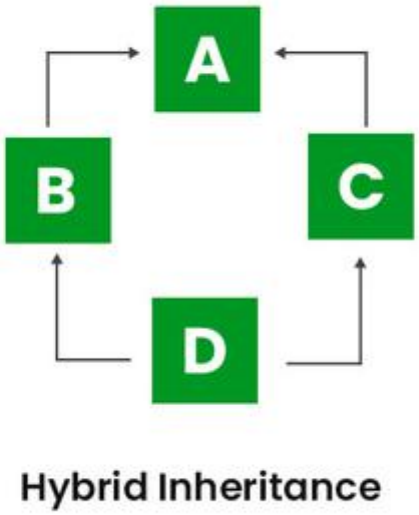
```
Geeks
for
Geeks
```



Multiple Inheritance

5. Hybrid Inheritance(Through Interfaces): It is a mix of two or more of the above types of inheritance. Since java doesn't support multiple inheritances

with classes, hybrid inheritance is also not possible with classes. In java, we can achieve hybrid inheritance only through Interfaces.



Hybrid Inheritance

Important facts about inheritance in Java

- **Default superclass**: Except Object class, which has no superclass, every class has one and only one direct superclass (single inheritance). In the absence of any other explicit superclass, every class is implicitly a subclass of the Object class.
- **Superclass can only be one:** A superclass can have any number of subclasses. But a subclass can have only **one** superclass. This is because Java does not support multiple inheritances with classes. Although with interfaces, multiple inheritances are supported by java.
- **Inheriting Constructors:** A subclass inherits all the members (fields, methods, and nested classes) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.
- **Private member inheritance:** A subclass does not inherit the private members of its parent class. However, if the superclass has public or protected methods(like getters and setters) for accessing its private fields, these can also be used by the subclass.

Java IS-A type of Relationship.

IS-A is a way of saying: This object is a type of that object. Let us see how the extends keyword is used to achieve inheritance.

- Java

```
public class SolarSystem {

}

public class Earth extends SolarSystem {

}

public class Mars extends SolarSystem {

}

public class Moon extends Earth {

}
```

Now, based on the above example, in Object-Oriented terms, the following are true:-

1. SolarSystem is the superclass of Earth class.
2. SolarSystem is the superclass of Mars class.
3. Earth and Mars are subclasses of SolarSystem class.
4. Moon is the subclass of both Earth and SolarSystem classes.

- Java

```
class SolarSystem {

}

class Earth extends SolarSystem {

}

class Mars extends SolarSystem {

}

public class Moon extends Earth {
```

```
    public static void main(String args[])
    {
        SolarSystem s = new SolarSystem();
        Earth e = new Earth();
        Mars m = new Mars();


        System.out.println(s instanceof SolarSystem);
        System.out.println(e instanceof Earth);
        System.out.println(m instanceof SolarSystem);
    }
}
```

Output

```
true
true
true
```

What all can be done in a Subclass?

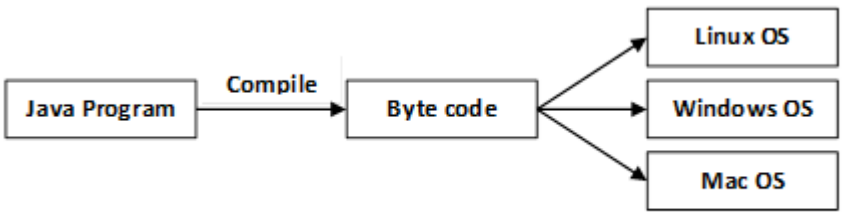
In sub-classes we can inherit members as is, replace them, hide them, or supplement them with new members:

- The inherited fields can be used directly, just like any other fields.
- We can declare new fields in the subclass that are not in the superclass.
- The inherited methods can be used directly as they are.
- We can write a new *instance* method in the subclass that has the same signature as the one in the superclass, thus <u>overriding</u> it (as in the example above, *toString()* method is overridden).
- We can write a new *static* method in the subclass that has the same signature as the one in the superclass, thus <u>hiding</u> it.
- We can declare new methods in the subclass that are not in the superclass.

- We can write a subclass constructor that invokes the constructor of the superclass, either implicitly or by using the keyword super.

Features of Java

- **Simple:** Java is a simple language because its syntax is simple, clean, and easy to understand. Complex and ambiguous concepts of C++ are either eliminated or re-implemented in Java. For example, pointer and operator overloading are not used in Java.
- **Object-Oriented:** In Java, everything is in the form of the object. It means it has some data and behavior. A program must have at least one class and object.
- **Robust:** Java makes an effort to check error at run time and compile time. It uses a strong memory management system called garbage collector. Exception handling and garbage collection features make it strong.
- **Secure:** Java is a secure programming language because it has no explicit pointer and programs runs in the virtual machine. Java contains a security manager that defines the access of Java classes.
- **Platform-Independent:** Java provides a guarantee that code writes once and run anywhere. This byte code is platform-independent and can be run on any machine.



- **Portable:** Java Byte code can be carried to any platform. No implementation-dependent features. Everything related to storage is predefined, for example, the size of primitive data types.
- **High Performance:** Java is an interpreted language. Java enables high performance with the use of the Just-In-Time compiler.
- **Distributed:** Java also has networking facilities. It is designed for the distributed environment of the internet because it supports TCP/IP protocol. It can run over the internet. EJB and RMI are used to create a distributed system.
- **Multi-threaded:** Java also supports multi-threading. It means to handle more than one job a time.

OOPs (Object Oriented Programming System)

Object-oriented programming is a way of solving a complex problem by breaking them into a small sub-problem. An object is a real-world entity. It is easier to develop a program by using an object. In OOPs, we create programs using class and object in a structured manner.

Class: A class is a template or blueprint or prototype that defines data members and methods of an object. An object is the instance of the class. We can define a class by using the class keyword.

Object: An object is a real-world entity that can be identified distinctly. For example, a desk, a circle can be considered as objects. An object has a unique behavior, identity, and state. Data fields with their current values represent the state of an object (also known as its properties or attributes).

Abstraction: An abstraction is a method of hiding irrelevant information from the user. For example, the driver only knows how to drive a car; there is no need to know how does the car run. We can make a class abstract by using the keyword abstract. In Java, we use abstract class and interface to achieve abstraction.

Encapsulation: An encapsulation is the process of binding data and functions into a single unit. A class is an example of encapsulation. In Java, Java bean is a fully encapsulated class.

Inheritance: Inheritance is the mechanism in which one class acquire all the features of another class. We can achieve inheritance by using the extends keyword. It facilitates the reusability of the code.

Polymorphism: The polymorphism is the ability to appear in many forms. In other words, single action in different ways. For example, a boy in the classroom behaves like a student, in house behaves like a son. There are two types of polymorphism: run time polymorphism and compile-time polymorphism.

Data Types in Java

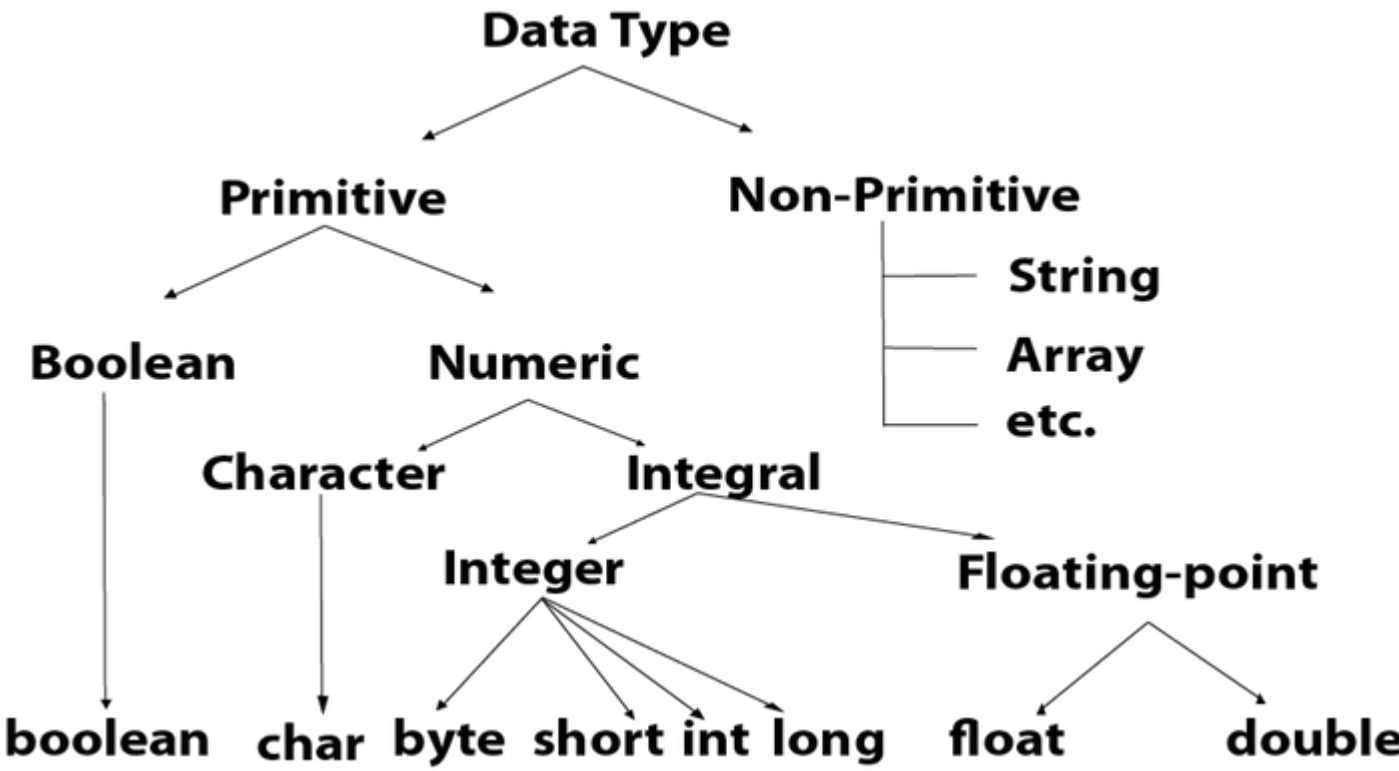
Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.

Java Primitive Data Types

Data Type	Default Value	Default size
Boolean	false	1 bit
Char	'\u0000'	2 byte
Byte	0	1 byte
Short	0	2 byte

- int data type
- long data type
- float data type
- double data type



Int	0	4 byte
Long	0L	8 byte
Float	0.0f	4 byte
Double	0.0d	8 byte

Boolean Data Type

The Boolean data type is used to store only two possible values: true and false. This data type is used for simple flags that track true/false conditions.

The Boolean data type specifies one bit of information, but its "size" can't be defined precisely.

Byte Data Type

The byte data type is an example of primitive data type. It isan 8-bit signed two's complement integer. Its value-range lies between -128 to 127 (inclusive). Its minimum value is -128 and maximum value is 127. Its default value is 0.

The byte data type is used to save memory in large arrays where the memory savings is most required. It saves space because a byte is 4 times smaller than an integer. It can also be used in place of "int" data type.

Short Data Type

The short data type is a 16-bit signed two's complement integer. Its value-range lies between -32,768 to 32,767 (inclusive). Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0.

The short data type can also be used to save memory just like byte data type. A short data type is 2 times smaller than an integer.

Int Data Type

The int data type is a 32-bit signed two's complement integer. Its value-range lies between -2,147,483,648 (-2^31) to 2,147,483,647 (2^31 -1) (inclusive). Its minimum value is -2,147,483,648and maximum value is 2,147,483,647. Its default value is 0.

The int data type is generally used as a default data type for integral values unless if there is no problem about mem

Long Data Type

The long data type is a 64-bit two's complement integer. Its value-range lies between -9,223,372,036,854,775,808(-2^63) to 9,223,372,036,854,775,807(2^63 -1)(inclusive). Its minimum value is - 9,223,372,036,854,775,808and maximum value is 9,223,372,036,854,775,807. Its default value is 0. The long data type is used when you need a range of values more than those provided by int.

Float Data Type

The float data type is a single-precision 32-bit IEEE 754 floating point.Its value range is unlimited. It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Double Data Type

The double data type is a double-precision 64-bit IEEE 754 floating point. Its value range is unlimited. The double data type is generally used for decimal values just like float. The double data type also should never be used for precise values, such as currency. Its default value is 0.0d.

Java Variables

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: local, instance and static.

There are two types of data types in Java: primitive and non-primitive.

Variable

A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.

Types of Variables

There are three types of variables in Java:

- local variable
- instance variable
- static variable

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

3) Static variable

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

Example to understand the types of variables in java

```
1. public class A
2. {
3.    static int m=100;//static variable
4.    void method()
5.    {
6.       int n=90;//local variable
7.    }
8.    public static void main(String args[])
9.    {
10.      int data=50;//instance variable
11.   }
12. }//end of class
```

Java Variable Example: Add Two Numbers

```
1. public class Simple{
2. public static void main(String[] args){
3. int a=10;
```

```
4. int b=10;
5. int c=a+b;
6. System.out.println(c);
7. }
8. }
```

Java Variable Example: Widening

```
1. public class Simple{
2. public static void main(String[] args){
3. int a=10;
4. float f=a;
5. System.out.println(a);
6. System.out.println(f);
7. }}
```

Java Variable Example: Narrowing (Typecasting)

```
1. public class Simple{
2. public static void main(String[] args){
3. float f=10.5f;
4. //int a=f;//Compile time error
5. int a=(int)f;
6. System.out.println(f);
7. System.out.println(a);
8. }}
```

Literals in Java

In Java, **literal** is a notation that represents a fixed value in the source code. In lexical analysis, literals of a given type are generally known as **tokens**. In this section, we will discuss the term **literals in Java**.

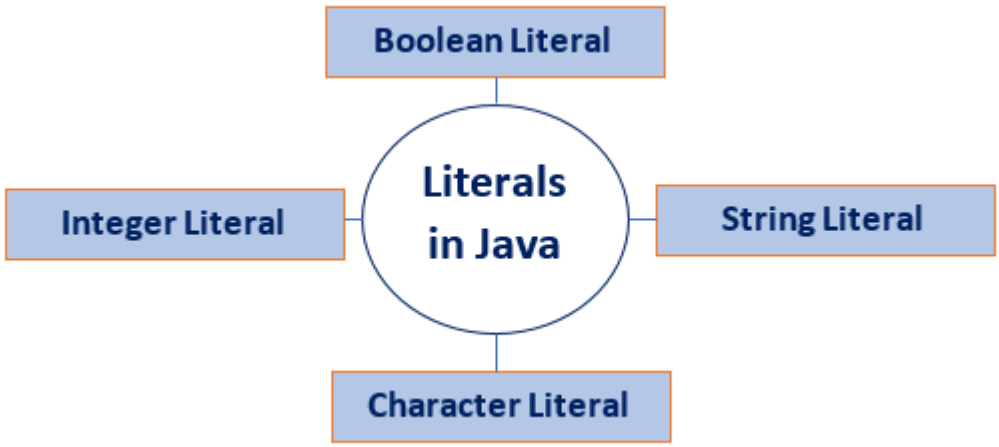
Literals

In Java, **literals** are the constant values that appear directly in the program. It can be assigned directly to a variable. Java has various types of literals. The following figure represents a literal.

Types of Literals in Java

There are the majorly **four** types of literals in Java:

1. Integer Literal
2. Character Literal
3. Boolean Literal
4. String Literal



Wrapper classes in Java

The **wrapper class in Java** provides the mechanism *to convert primitive into object and object into primitive*.

Since J2SE 5.0, **autoboxing** and **unboxing** feature convert primitives into objects and objects into primitives automatically. The automatic conversion of primitive into an object is known as autoboxing and vice-versa unboxing.

Use of Wrapper classes in Java

Java is an object-oriented programming language, so we need to deal with objects many times like in Collections, Serialization, Synchronization, etc. Let us see the different scenarios, where we need to use the wrapper classes.

- **Change the value in Method:** Java supports only call by value. So, if we pass a primitive value, it will not change the original value. But, if we convert the primitive value in an object, it will change the original value.
- **Serialization:** We need to convert the objects into streams to perform the serialization. If we have a primitive value, we can convert it in objects through the wrapper classes.
- **Synchronization:** Java synchronization works with objects in Multithreading.
- **java.util package:** The java.util package provides the utility classes to deal with objects.
- **Collection Framework:** Java collection framework works with objects only. All classes of the collection framework (ArrayList, LinkedList, Vector, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, etc.) deal with objects only.

The eight classes of the *java.lang* package are known as wrapper classes in Java. The list of eight wrapper classes are given below

Primitive Type	Wrapper class
boolean	Boolean
Char	Character
Byte	Byte
Short	Short
Int	Integer
Long	Long
Float	Float
Double	Double

Autoboxing

The automatic conversion of primitive data type into its corresponding wrapper class is known as autoboxing, for example, byte to Byte, char to Character, int to Integer, long to Long, float to Float, boolean to Boolean, double to Double, and short to Short.

Since Java 5, we do not need to use the valueOf() method of wrapper classes to convert the primitive into objects.

Wrapper class Example: Primitive to Wrapper

```
//Java program to convert primitive into objects
//Autoboxing example of int to Integer
public class WrapperExample1{
public static void main(String args[]){
//Converting int into Integer
int a=20;
Integer i=Integer.valueOf(a);//converting int into Integer explicitly
Integer j=a;//autoboxing, now compiler will write Integer.valueOf(a) internally

System.out.println(a+" "+i+" "+j);
}}
```

Unboxing

The automatic conversion of wrapper type into its corresponding primitive type is known as unboxing. It is the reverse process of autoboxing. Since Java 5, we do not need to use the intValue() method of wrapper classes to convert the wrapper type into primitives.

Wrapper class Example: Wrapper to Primitive

```
//Java program to convert object into primitives
//Unboxing example of Integer to int
public class WrapperExample2{
public static void main(String args[]){
//Converting Integer to int
Integer a=new Integer(3);
```

```
7. int i=a.intValue();//converting Integer to int explicitly
8. int j=a;//unboxing, now compiler will write a.intValue() internally
9.
10. System.out.println(a+" "+i+" "+j);
11. }}
```

Custom Wrapper class in Java

Java Wrapper classes wrap the primitive data types, that is why it is known as wrapper classes. We can also create a class which wraps a primitive data type. So, we can create a custom wrapper class in Java.

Java Arrays

Normally, an array is a collection of similar type of elements which has contiguous memory location.

Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.

In Java, array is an object of a dynamically generated class. Java array inherits the Object class, and implements the Serializable as well as Cloneable interfaces. We can store primitive values or objects in an array in Java. Like C/C++, we can also create single dimentional or multidimentional arrays in Java.

Types of Array in java

There are two types of array.

- Single Dimensional Array
- Multidimensional Array

Single Dimensional Array in Java

Syntax to Declare an Array in Java

1. dataType[] arr; (or)
2. dataType []arr; (or)
3. dataType arr[];

Instantiation of an Array in Java

1. arrayRefVar=**new** datatype[size];

```
/Java Program to illustrate how to declare, instantiate, initialize
//and traverse the Java array.

class Testarray{
public static void main(String args[]){
int a[]=new int[5];//declaration and instantiation
a[0]=10;//initialization
a[1]=20;
a[2]=70;
a[3]=40;
a[4]=50;
//traversing array
for(int i=0;i<a.length;i++)//length is the property of array
System.out.println(a[i]);
}}
```

Declaration, Instantiation and Initialization of Java Array

We can declare, instantiate and initialize the java array together by:

int a[]={33,3,4,5};//declaration, instantiation and initialization

```
//Java Program to illustrate the use of declaration, instantiation
//and initialization of Java array in a single line
class Testarray1{
public static void main(String args[]){
int a[]={33,3,4,5};//declaration, instantiation and initialization
//printing array
for(int i=0;i<a.length;i++)//length is the property of array
System.out.println(a[i]);
}}
```

For-each Loop for Java Array

We can also print the Java array using **for-each loop**. The Java for-each loop prints the array elements one by one. It holds an array element in a variable, then executes the body of the loop.

The syntax of the for-each loop is given below:

```
for(data_type variable:array){
//body of the loop
    }
```

```
1. //Java Program to print the array elements using for-each loop
2. class Testarray1{
3. public static void main(String args[]){
4. int arr[]={33,3,4,5};
5. //printing array using for-each loop
6. for(int i:arr)
7. System.out.println(i);
8. }}
```

Passing Array to a Method in Java

We can pass the java array to method so that we can reuse the same logic on any array.

Let's see the simple example to get the minimum number of an array using a method.

```
1. //Java Program to demonstrate the way of passing an array
2. //to method.
3. class Testarray2{
4. //creating a method which receives an array as a parameter
5. static void min(int arr[]){
6. int min=arr[0];
7. for(int i=1;i<arr.length;i++)
8.  if(min>arr[i])
9.   min=arr[i];
10.
```

```
System.out.println(min);
}

public static void main(String args[]){
int a[]={33,3,4,5};//declaring and initializing an array
min(a);//passing array to method
}}
```

Java Arithmetic Operators

Operators constitute the basic building block to any programming language. Java too provides many types of operators which can be used according to the need to perform various calculations and functions, be it logical, arithmetic, relational, etc. They are classified based on the functionality they provide. Here are a few types:

1. Arithmetic Operators
2. Unary Operators
3. Assignment Operator
4. Relational Operators
5. Logical Operators
6. Ternary Operator
7. Bitwise Operators
8. Shift Operators

This article explains all that one needs to know regarding Arithmetic Operators.

Arithmetic Operators

These operators involve the mathematical operators that can be used to perform various simple or advanced arithmetic operations on the primitive data types referred to as the operands. These operators consist of various unary and binary operators that can be applied on a single or two operands. Let's look at the various operators that Java has to provide under the arithmetic operators.

Operators	Result
+	Addition of two numbers
-	Subtraction of two numbers
*	Multiplication of two numbers
/	Division of two numbers
%	(Modulus Operator)Divides two numbers and returns the remainder

Unary Operator

Operators constitute the basic building block to any programming language. Java too provides many types of operators which can be used according to the need to perform various calculations and functions be it logical, arithmetic, relational, etc. They are classified based on the functionality they provide. Here are a few types:

1. Arithmetic Operators
2. Unary Operators
3. Assignment Operator
4. Relational Operators
5. Logical Operators
6. Ternary Operator
7. Bitwise Operators
8. Shift Operators

Types of Assignment Operators in Java

The Assignment Operator is generally of two types. They are:

1. Simple Assignment Operator: The Simple Assignment Operator is used with the "=" sign where the left side consists of the operand and the right side consists of a value. The value of the right side must be of the same data type that has been defined on the left side.

2. Compound Assignment Operator: The Compound Operator is used where +,-,*, and / is used along with the = operator.

Let's look at each of the assignment operators and how they operate:

1. (=) operator:

This is the most straightforward assignment operator, which is used to assign the value on the right to the variable on the left. This is the basic definition of an assignment operator and how it functions.

Syntax:

```
num1 = num2;
```

Example:

```
a = 10;

ch = 'y';
```

Java Classes/Objects

Java is an object-oriented programming language.

Everything in Java is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an object. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.

A Class is like an object constructor, or a "blueprint" for creating objects.

Create a Class

To create a class, use the keyword `class`:

Main.java

Create a class named "`Main`" with a variable x:

```
public class Main {

  int x = 5;
```

```
}
```

Create an Object

In Java, an object is created from a class. We have already created the class named `Main`, so now we can use this to create objects.

To create an object of `Main`, specify the class name, followed by the object name, and use the keyword `new`:

Example

Create an object called "`myObj`" and print the value of x:

```
public class Main {

  int x = 5;



  public static void main(String[] args) {

    Main myObj = new Main();

    System.out.println(myObj.x);

  }
```

```
}
```

Java Class Attributes

we used the term "variable" for `x` in the example (as shown below). It is actually an **attribute** of the class. Or you could say that class attributes are variables within a class:

Example

Create a class called "`Main`" with two attributes: `x` and `y`:

```
public class Main {

  int x = 5;

  int y = 3;

}
```

Accessing Attributes

You can access attributes by creating an object of the class, and by using the dot syntax (`.`):

The following example will create an object of the `Main` class, with the name `myObj`. We use the `x` attribute on the object to print its value:

Example

Create an object called "`myObj`" and print the value of `x`:

```
public class Main {

  int x = 5;



  public static void main(String[] args) {

    Main myObj = new Main();

    System.out.println(myObj.x);

  }

}
```

You can also modify attribute values:

Example

Set the value of `x` to 40:

```
public class Main {

  int x;



  public static void main(String[] args) {

    Main myObj = new Main();

    myObj.x = 40;

    System.out.println(myObj.x);

  }

}
```

What is Exception Handling?

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.

Advantage of Exception Handling

The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

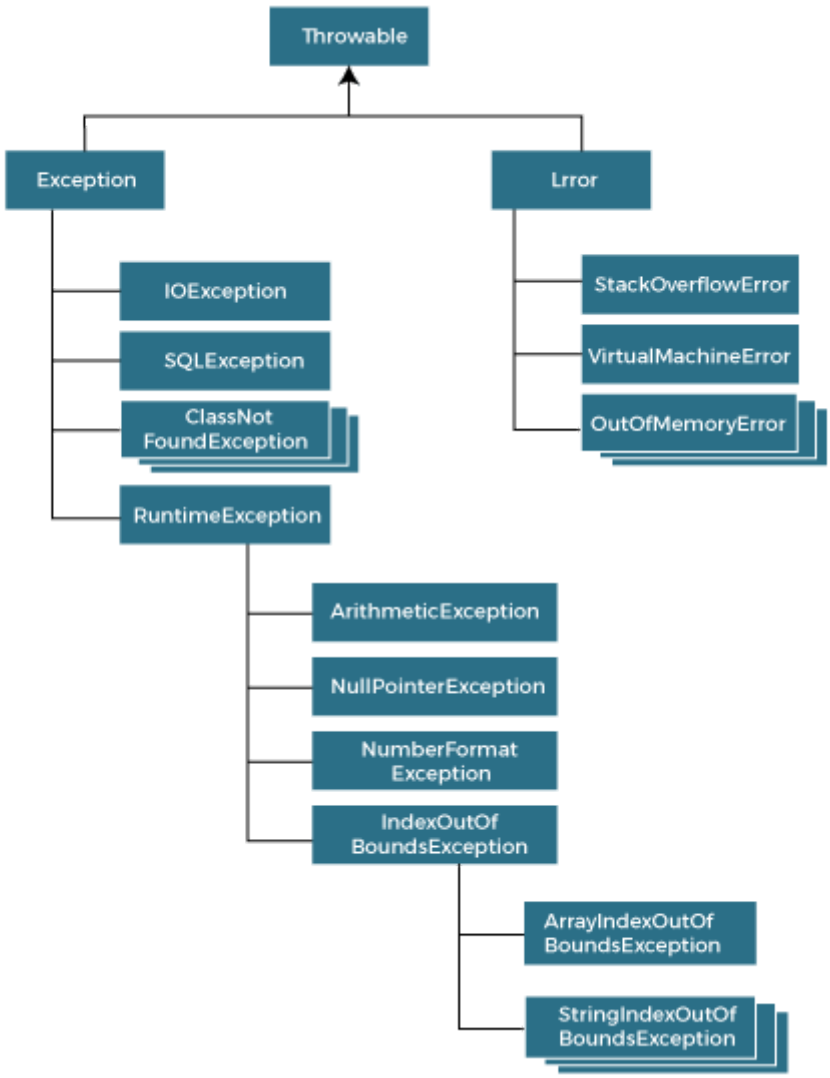
1. statement 1;
2. statement 2;
3. statement 3;
4. statement 4;
5. statement 5;//exception occurs
6. statement 6;

7. statement 7;
8. statement 8;
9. statement 9;
10. statement 10;

Suppose there are 10 statements in a Java program and an exception occurs at statement 5; the rest of the code will not be executed, i.e., statements 6 to 10 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in Java.

Hierarchy of Java Exception classes

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error

Difference between Checked and Unchecked Exceptions

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException,

ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Java Exception Handling Example

Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

JavaExceptionExample.java

```
1. public class JavaExceptionExample{
2.   public static void main(String args[]){
3.    try{
4.       //code that may raise exception
5.       int data=100/0;
6.    }catch(ArithmeticException e){System.out.println(e);}
7.    //rest code of the program
8.    System.out.println("rest of the code...");
9.   }
10.}
```

Difference between throw and throws in Java

The throw and throws is the concept of exception handling where the throw keyword throw the exception explicitly from a method or a block of code whereas the throws keyword is used in signature of the method.

There are many differences between throw and throws keywords. A list of differences between throw and throws are given below:

Sr. no.	Basis of Differences	throw	throws
1.	Definition	Java throw keyword is used throw an exception explicitly in the code, inside the function or the block of code.	Java throws keyword is used in the method signature to declare an exception which might be thrown by the function while the execution of the code.
2.	Type of exception Using throw keyword, we can only propagate unchecked exception i.e., the checked exception cannot be propagated using throw only.	Using throws keyword, we can declare both checked and unchecked exceptions. However, the throws keyword can be used to propagate checked exceptions only.	
3.	Syntax	The throw keyword is followed by an instance of Exception to be thrown.	The throws keyword is followed by class names of Exceptions to be thrown.
4.	Declaration	throw is used within the method.	throws is used with the method signature.
5.	Internal implementation	We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.	We can declare multiple exceptions using throws keyword that can be thrown by the method. For example, main() throws IOException, SQLException.

Java throw Example

TestThrow.java

```
1. public class TestThrow {
2.     //defining a method
3.     public static void checkNum(int num) {
4.         if (num < 1) {
5.             throw new ArithmeticException("\nNumber is negative, cannot calculate square");
6.         }
```

```
7.          else {
8.              System.out.println("Square of " + num + " is " + (num*num));
9.          }
10.     }
11.     //main method
12.     public static void main(String[] args) {
13.             TestThrow obj = new TestThrow();
14.             obj.checkNum(-3);
15.             System.out.println("Rest of the code..");
16.     }
17. }
```

Java Applet

Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet

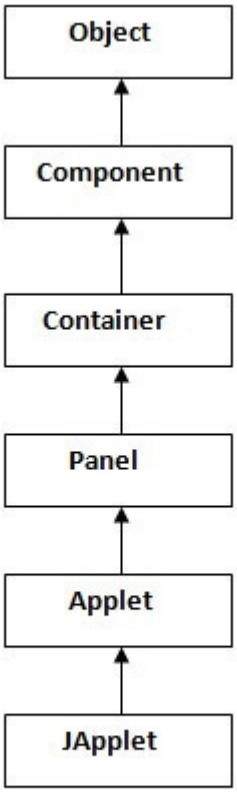
There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many plateforms, including Linux, Windows, Mac Os etc.

Drawback of Applet

- Plugin is required at client browser to execute applet.

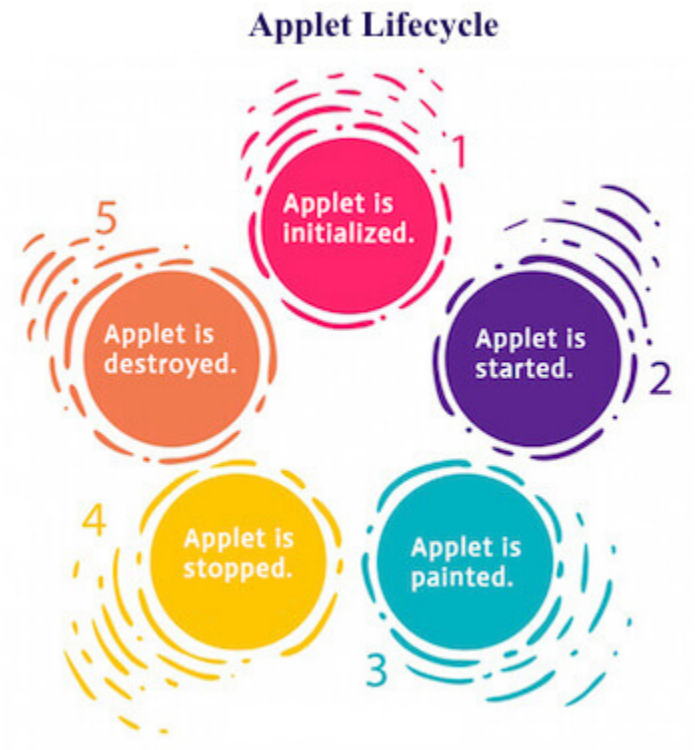
Hierarchy of Applet



As displayed in the above diagram, Applet class extends Panel. Panel class extends Container which is the subclass of Component.

Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.



Lifecycle methods for Applet:

The java.applet.Applet class 4 life cycle methods and java.awt.Component class provides 1 life cycle methods for an applet.

java.applet.Applet class

For creating any applet java.applet.Applet class must be inherited. It provides 4 life cycle methods of applet

1. **public void init():** is used to initialized the Applet. It is invoked only once.
2. **public void start():** is invoked after the init() method or browser is maximized. It is used to start the Applet.
3. **public void stop():** is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
4. **public void destroy():** is used to destroy the Applet. It is invoked only once.

java.awt.Component class

The Component class provides 1 life cycle method of applet.

1. **public void paint(Graphics g):** is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

How to run an Applet?

There are two ways to run an applet

1. By html file.
2. By appletViewer tool (for testing purpose).

Simple example of Applet by html file:

To execute the applet by html file, create an applet and compile it. After that create an html file and place the applet code in html file. Now click the html file.

```
1. //First.java
2. import java.applet.Applet;
3. import java.awt.Graphics;
4. public class First extends Applet{
5.
6. public void paint(Graphics g){
7. g.drawString("welcome",150,150);
8. }
9.
10. }
```

myapplet.html

```
1. <html>
2. <body>
3. <applet code="First.class" width="300" height="300">
4. </applet>
5. </body>
6. </html>
```

Simple example of Applet by appletviewer tool:

To execute the applet by appletviewer tool, create an applet that contains applet tag in comment and compile it. After that run it by: appletviewer First.java. Now Html file is not required but it is for testing purpose only.

```
1. //First.java
2. import java.applet.Applet;
3. import java.awt.Graphics;
```

```
4. public class First extends Applet{
5.
6. public void paint(Graphics g){
7. g.drawString("welcome to applet",150,150);
8. }
9.
10.}
11./*
12.<applet code="First.class" width="300" height="300">
13.</applet>
14.*/
```

To execute the applet by appletviewer tool, write in command prompt:

```
c:\>javac First.java
c:\>appletviewer First.java
```

Displaying Graphics in Applet

java.awt.Graphics class provides many methods for graphics programming.

Commonly used methods of Graphics class:

1. **public abstract void drawString(String str, int x, int y):** is used to draw the specified string.
2. **public void drawRect(int x, int y, int width, int height):** draws a rectangle with the specified width and height.
3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.
5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.

8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.
9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.
10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.
11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

Example of Graphics in applet:

```
import java.applet.Applet;
import java.awt.*;

public class GraphicsDemo extends Applet{

public void paint(Graphics g){
g.setColor(Color.red);
g.drawString("Welcome",50, 50);
g.drawLine(20,30,20,300);
g.drawRect(70,100,30,30);
g.fillRect(170,100,30,30);
g.drawOval(70,200,30,30);

g.setColor(Color.pink);
g.fillOval(170,200,30,30);
g.drawArc(90,150,30,30,30,270);
g.fillArc(270,150,30,30,0,180);

}
}
```

myapplet.html

```
<html>
<body>
```

```
3. <applet code="GraphicsDemo.class" width="300" height="300">
4. </applet>
5. </body>
6. </html>
```